Mechanical Engineering Skills in Claude

Achuth Chandrasekhar* Jesse Barkley* Peter Pak* Amir Barati Farimani†

Department of Mechanical Engineering, Carnegie Mellon University, Pittsburgh, PA 15213, USA

*Equal contribution †Corresponding author: barati@cmu.edu

Abstract

Mechanical engineers rely on dozens of specialized tools—CAD kernels, finite-element and CFD solvers, flight simulators, process maps, thermodynamic databases and research models—each with its own interface, installation and learning curve. We present *Mechanical Engineering Skills*, an open library of 32 agent skills for design, analysis and manufacturing that makes this breadth available through a single conversation in Claude, by text or by voice. Each skill is a folder that pairs concise operating instructions with tested scripts: the language model understands the request, selects the skill and orchestrates the work, while deterministic tools do the computation, and every result is returned with the check that supports it. Users do not need to learn each tool, train a model, or download and run models on their own computer; skills are loaded only when a request needs them and install with one command. Nineteen skills package published or preprint research from our laboratory, spanning text-to-CAD, linkage synthesis, CFD case setup, stress-field and lattice surrogates, equation discovery, metal and polymer additive-manufacturing process planning, and in-process monitoring. We describe the skill format, the design principles, the procedure we used to turn a paper into a reliable skill, and case studies in which the skills reproduce the source papers: rocket apogee within 4% of OpenRocket, exact lattice homogenization on solvable cases, an anisotropic lattice designed to a target stiffness in two minutes on a laptop, and process suggestions that match operators' choices batch by batch. The library is available at https://github.com/BaratiLab/mechanical-engineering-skills.

1 Introduction

Mechanical engineering work is a chain of small, checkable steps: read a requirement, pick a model, compute, compare against a hand estimate or a known case, and decide. General-purpose language-model agents are now capable of driving such chains: they write CAD scripts, set up simulations, and read sensor data. Our own work has shown this across the life cycle of a part, from text-to-CAD [1, 2] and mechanism synthesis [3] to LLM-in-the-loop structural design [4], autonomous CFD [5], additive-manufacturing process planning [6–8] and print monitoring [9].

Two obstacles keep this research from becoming everyday practice. First, each method lives in its own code base, with its own environment, data and conventions, and is rarely packaged for someone who simply has an engineering question. Second, agents left to themselves make the kind of errors engineers cannot accept: arithmetic slips, wrong units, a correlation used outside its range, or a confident answer with no evidence behind it.

Agent Skills [10, 11] offer a lightweight remedy. A skill is a directory with a `SKILL.md` file—a short description that tells the agent when to use it, followed by instructions—plus any scripts and reference files the instructions point to. The agent sees only the one-line descriptions until a task matches, then loads the instructions, and reads deeper files or runs scripts only when needed (*progressive disclosure*). Skills are portable across Claude Code, Claude.ai and other agents that adopted the format, and they are installed with a single command.

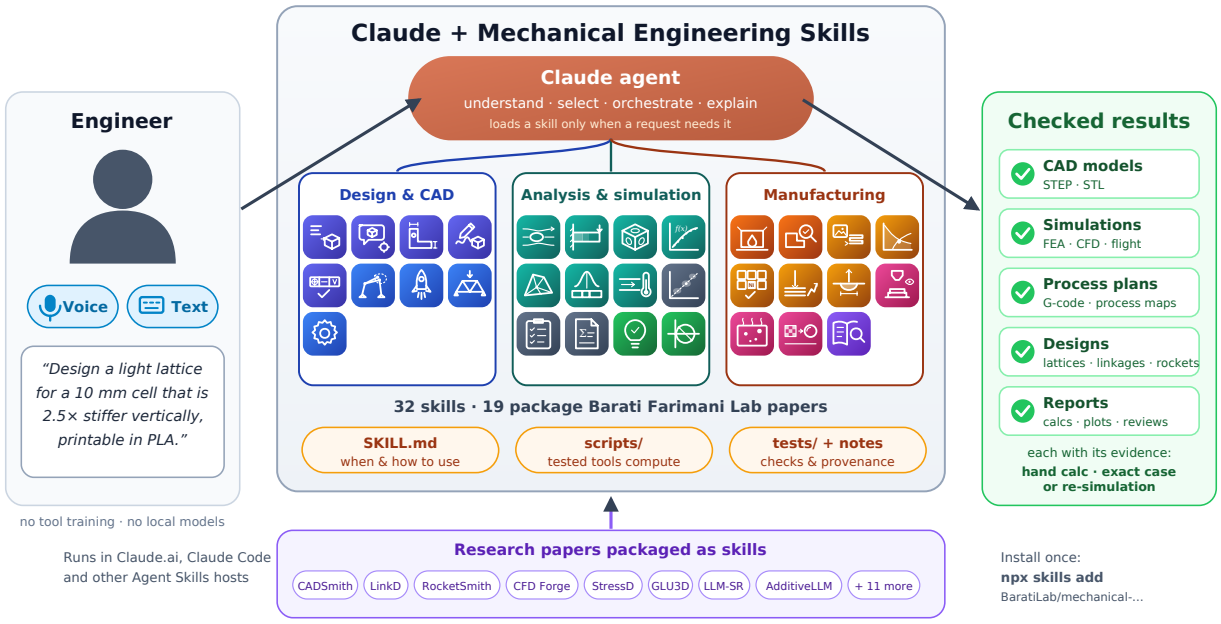


Figure 1: Mechanical Engineering Skills in Claude. An engineer states a goal by voice or text. The Claude agent understands the request, selects the relevant skills from a library of 32 (design and CAD, analysis and simulation, manufacturing), and orchestrates them. Each skill bundles instructions (`SKILL.md`), tested scripts that do the computation, and checks. The results—CAD models, simulations, process plans, designs and reports—come back with the evidence that supports them. Nineteen skills package research papers from our laboratory. No per-tool training or locally hosted model is required.

This paper presents *Mechanical Engineering Skills* [12], a library of 32 such skills (Fig. 1, Table 2). Our contributions are:

1. **A library** covering design, analysis and manufacturing, in which 19 skills package peer-reviewed or preprint research from our laboratory and 13 cover core engineering tasks (machine-element sizing, FEA with hand checks, tolerance stack-ups, material selection, design reviews and teaching aids).
2. **Design principles** for engineering skills: tools compute and the model orchestrates; every number carries its check; the paper's protocol is reproduced but its weaknesses are corrected and declared; and the skill states the boundary of its validity.
3. **Access without expertise in each tool**: the whole library is reachable by text or voice in one conversation, without learning tool interfaces, training models or running models locally (§4).
4. **A procedure for turning a paper into a skill**, applied to 19 papers, with case studies that reproduce their key results.

2 Related work

Language-model agents and tool use. Large language models acquire broad knowledge and in-context reasoning from scale [13, 14], and become agents when they interleave reasoning with actions and observations [15] or learn to call external tools [16]; surveys catalogue the resulting architectures [17]. Retrieval augments generation with documents the model was not trained

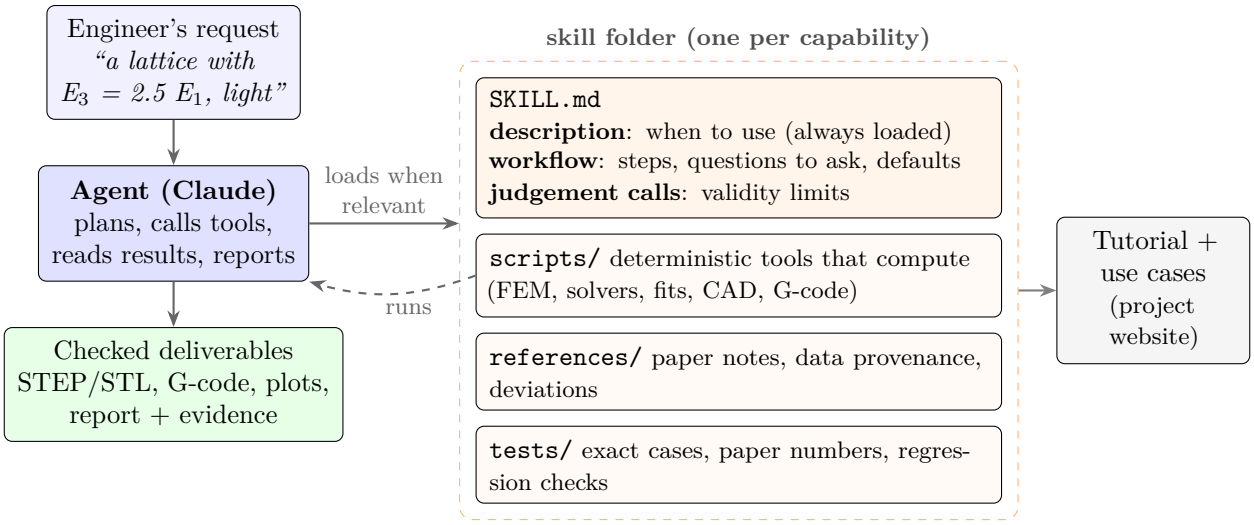


Figure 2: Anatomy of an engineering skill and how an agent uses it. Only the one-line description is always in the agent's context; instructions, scripts and references are loaded on demand. Tests and a public tutorial accompany every research skill.

on [18], and the Model Context Protocol standardizes connections to external systems [19]. Agent Skills [10, 11] add a complementary layer: packaged procedural knowledge and code that an agent loads only when relevant. In science, tool-using agents have planned and executed chemistry experiments [20, 21]; our library brings the same pattern to mechanical engineering, with an emphasis on checks.

Language models in mechanical design and manufacturing. Early studies showed that general models can help across the design-to-manufacture pipeline but make errors that require verification [22]. Subsequent work generated CAD from text with execution feedback [1, 2], coupled models with finite-element analysis for structural design [4], automated CFD setup under physics constraints [5], and applied agents to additive manufacturing, from literature question answering [23] and defect prediction [6] to alloy evaluation [7], process optimization [8], print monitoring and control [9] and complete design-to-flight workflows [24]. These systems are typically bespoke; this work packages them in a common, portable format.

Learned surrogates and physics models. Generative models, in particular denoising diffusion [25–28], now produce stress fields [29], lattice architectures [30], mechanisms [3], toolpaths [31] and super-resolved process images [32], often with U-Net backbones [33]. Machine learning also predicts melt-pool geometry [34] and porosity [35] in metal additive manufacturing [36, 37], and symbolic regression recovers interpretable laws from data [38–40]. The skills pair such models with classical references—Rosenthal's moving-source solution [41], Barrowman's stability method [42], open-source solvers [43, 44], computational thermodynamics [45], cellular-solid scaling [46, 47], numerical homogenization [48] and Bayesian optimization [49]—so that every learned prediction can be checked against physics.

3 Skills as a unit of engineering capability

Figure 2 shows the structure we use for every skill. The `SKILL.md` front matter carries a description written for the agent's routing decision: what the skill does, what it is based on, and the situations that should trigger it. The body is a short workflow—typically four to six numbered steps—with the commands to run and, crucially, the *judgement calls* an experienced engineer would make: which inputs to ask for, what to report alongside the headline number, and where the method stops being valid. Long material (paper notes, provenance, interpretation choices) sits in `references/` so that it costs no context unless needed.

The `scripts/` directory holds the computation. Scripts are ordinary command-line Python programs with JSON output so that the agent can parse results reliably. Examples include a periodic voxel homogenizer for lattices, a plane-stress Q4 finite-element solver, a Barrowman stability and one-dimensional flight simulator for rockets, a Rosenthal melt-pool solver, an SQLite build log with Gaussian-process suggestions for selective laser sintering, and validators for OpenFOAM cases. Each research skill also ships a test suite (`tests/`) and a tutorial on the project website that walks through the paper's main experiment with real outputs.

4 Engineering on demand, by voice or text

The practical value of a skill library is that it removes the cost of *reaching* a capability. Without it, using a research method or a specialized analysis means finding the code, installing its environment, learning its inputs and conventions, often downloading or training a model and providing a GPU, and then interpreting outputs correctly. With skills, the user states the goal and the agent does the rest.

One conversation, many tools. All 32 skills are installed with a single command and are available in the same conversation. Only each skill's one-line description is kept in the model's context (on the order of a hundred tokens per skill), so breadth costs almost nothing until a skill is used. The agent matches the request to a description, loads that skill's instructions, and runs its scripts. Skills compose naturally: a lattice designed with `lattice-inverse-design` can be checked for overhangs with `agentscad-fdm-dfam` and turned into G-code without the user switching tools.

No training on each tool. The instructions in a skill encode what an expert user of the underlying method knows: which inputs matter, what defaults are safe, which command to run and how to read its output. A user therefore does not need to know the command-line options of a homogenizer, the input format of a flight simulator, or the conventions of a thermodynamic database. The agent asks only for the few inputs that change the answer—for example the printed modulus of the material or the minimum wall thickness—and reports results in engineering terms.

By text or by voice. Because the interface is natural language, the same skills are available wherever Claude is: typed in Claude.ai or Claude Code, or spoken through voice mode in the Claude apps. An engineer at a printer or on the shop floor can ask, hands-free, whether a parameter set risks lack of fusion, or what fin span a rocket needs, and receive the answer with its supporting check. Figure 3 shows a typical exchange.

No models to download or run locally. The language model runs in the cloud, and the skills themselves are text and lightweight scripts. Where a source paper relied on a large learned model,

Table 1: Representative requests by voice or text, the skills the agent uses, and what is returned.

Request (voice or text)	Skills used	What comes back
"Size the shaft and bearings for a 2 kW gearbox output at 300 rpm."	`machine-element-sizing`	Diameters, bearing selection and fatigue safety factors with the calculation shown
"Make me a bracket for an M6 bolt pattern, 3 mm wall, printable without supports."	`cadsmith-text-to-cad`, `agentscad-fdm-dfam`	Validated STEP/STL, reoriented, with teardrop holes and an overhang report
"Is this aluminium part stiff enough? Check it under a 500 N tip load."	`quick-fea`	Deflection, peak stress and modes, compared against a beam hand calculation
"Set up flow over this heat-sink fin array at 2 m/s and tell me if the result is converged."	`cfd-simulation-setup`, `thermal-fluids-calc`	OpenFOAM case, run diagnosis and an accept/rerun decision tied to physics checks
"Design a lattice infill 2.5× stiffer vertically, as light as possible."	`lattice-inverse-design`	Cell choice, full stiffness matrix, weakest-direction modulus and tiled STL
"Will 280 W at 900 mm/s keyhole on Ti-6Al-4V?"	`lpbf-defect-prediction`, `meltpool-geometry`	Predicted regime, melt-pool size and a process map marking the safe window
"Layer 140 looks odd on the camera—what's wrong and what should I change?"	`llm-3d-print-monitor`	Diagnosis from the layer image and bounded G-code corrections for approval
"Which layers of yesterday's build should go to CT first?"	`thermal-porosity`	Per-layer pore counts and maps from the pyrometry, ranked for inspection
"Find a law for this stress–strain data that still works at higher strain."	`equation-discovery`	An interpretable equation with in-domain and out-of-domain error
"Give my students three Mohr's-circle problems with worked solutions."	`worked-problem-generator`, `mechanics-visuals`	Problems, code-verified solutions and figures

the skill's default path uses physics models, solvers or small fitted models that run in seconds on a CPU—for example search with exact homogenization instead of a 3D diffusion model for one-off lattice designs, or a Rosenthal solution calibrated on a few tracks instead of a trained network for melt-pool size. In Claude.ai the scripts run in Claude's own sandboxed code-execution environment, so nothing needs to be installed on the user's computer; in Claude Code they run in the user's project. Training the larger research models remains possible for users who want it, but it is never required to get an answer.

Table 1 lists representative requests, spoken or typed, with the skills the agent chains and what it returns. In each case the user describes the engineering goal; the choice of tools, their inputs and the checks are handled by the skills.

5 Design principles

P1. The model orchestrates; tools compute. Every number a user might act on is produced by code, never by the model's arithmetic. RocketSmith [24] reached the same conclusion after flying four agent-designed rockets; we apply it library-wide. The model remains responsible for what it is good at: eliciting requirements, choosing a method, chaining tools, and explaining results.

P2. Every result carries its check. Skills return the evidence for a result alongside it: a hand calculation for an FEA run, the exact laminate solution before trusting a lattice homogenization, re-simulation of a surrogate's best candidate, a kernel validity and volume check after a CAD edit. Where a published method used a learned scorer for acceptance (e.g. a surrogate predicting a generated lattice's modulus), the skill uses the physics model for acceptance and keeps the learned

Engineer (voice): *"I need a lattice infill for a 10 mm cell that is about two and a half times stiffer vertically than sideways, as light as possible, printed in PLA."*

Agent: loads `lattice-inverse-design`; asks one question (minimum wall the printer can hold); converts the target to $E_1 = E_2 \approx 0.06\,E_s$, $E_3 \approx 0.15\,E_s$; searches homogenized cells and refines thickness; re-checks the best cell at higher resolution.

Agent (reply): *"A gyroid–diamond hybrid with two periods sideways gives $E_3/E_1 = 2.6$ at 26% density (about 0.2 and 0.5 GPa in PLA). Its weakest direction is still $0.057\,E_s$, so it does not go soft off-axis. The tiled $3 \times 3 \times 3$ STL is attached."*

Figure 3: A typical exchange. The engineer names the goal; the skill supplies the procedure, the computation and the check. No tool-specific input or local model is needed.

model for guidance.

P3. Reproduce the paper, then fix what would mislead a user. Each research skill implements the paper's protocol so that its headline numbers can be reproduced, and then changes defaults where the original choice would mislead a practitioner, for example by checking units on load or by accepting generated designs only after re-simulation. Every deviation is listed in the skill's paper notes with the reason and the evidence.

P4. State the boundary. Skills say what they do not do: elastic homogenization is not crash performance; a flight simulation is a design check, not a launch approval; a defect classifier trained on one machine family is not a process qualification. These statements live in the instructions so the agent repeats them at the point of use.

P5. Ask once, then act. Instructions list the few inputs that change the answer (a material's printed modulus, minimum wall thickness, certification level) and sensible defaults for the rest, so that the agent asks one focused question rather than many generic ones.

6 The library

Table 2 lists all 32 skills, grouped into design and CAD, analysis and simulation, manufacturing, and teaching. Each row gives the skill's icon and name and, in plain language, what a user can ask for. Nineteen skills package research from our laboratory; the remaining 13 cover core engineering tasks that every practitioner needs, from machine-element sizing and FEA with hand checks to tolerance stack-ups, material selection [50] and design reviews, plus two skills for teaching.

Table 2: The skill library. Every skill is installed with one command and used by asking for the result in plain language; the agent selects the skill from its description. A citation marks the 19 skills that package a Barati Farimani Lab paper.

Skill	What you can ask for	Paper
Design and CAD		
`cadsmith-text-to-cad`	Describe a part in words; get a CadQuery model checked by exact geometry measurements and a three-view review, exported as STEP/STL.	[1]

	Skill	What you can ask for	Paper
	`query2cad-freecad`	Plain-language request to a FreeCAD macro, with automatic error repair and a visual self-check.	[2]
	`parametric-part-cad`	Part specification in plain English to a parametric CadQuery script, STEP/STL and a preview.	–
	`sketch-to-cad`	Hand sketch, photo or video frames to a dimensioned CAD model.	–
	`drawing-gdt-review`	Audit an engineering drawing for missing dimensions and GD&T errors.	–
	`linkage-synthesis`	Target motion path to a four- or six-bar linkage, built joint by joint and checked kinematically.	[3]
	`rocketsmith`	Hobby high-power rocket: stability, fin sizing, flight prediction, drag calibration and printable parts.	[24]
	`llm-mechanical-designer`	Truss design loop: the model proposes, exact FEM evaluates, the best designs are ranked.	[4]
	`machine-element-sizing`	Bolts, shafts, bearings, springs, gears and welds, with the work shown and safety factors.	–
Analysis and simulation			
	`cfd-simulation-setup`	Flow problem to an OpenFOAM case, run and diagnosis; results accepted only against physics checks.	[5]
	`stress-field-surrogate`	Geometry, supports and loads to a full stress field in milliseconds, with its own FEM data generator.	[29]
	`lattice-inverse-design`	Target stiffness to a printable TPMS or hybrid lattice cell with its full stiffness matrix and a tiled STL.	[30]
	`equation-discovery`	Test data to an interpretable equation that extrapolates beyond the data.	[40]
	`quick-fea`	Linear static and modal FEA with a required hand-calculation check.	–
	`tolerance-stackup`	Worst-case, RSS and Monte Carlo tolerance stack-ups with contributor charts.	–
	`thermal-fluids-calc`	Pipe pressure drop, heat exchangers and fins with real fluid properties.	–
	`material-selection`	Performance indices to a ranked material shortlist with Ashby charts.	–
	`design-review-package`	Requirements, FMEA, bill-of-materials cost roll-up and review checklist.	–
	`engineering-calc-report`	Turn calculations into a calculation report ready for sign-off.	–
Manufacturing			

	Skill	What you can ask for	Paper
	`agentscad-fdm-dfam`	Make a STEP part printable by FDM: reorientation, teardrop holes and chamfers, with verified geometry.	[51]
	`dfm-review`	Design-for-manufacture review for machining, sheet metal, molding and additive manufacturing.	–
	`image2gcode`	Photo or sketch to printer-ready G-code for 2D and 2.5D parts.	[31]
	`lpbf-defect-prediction`	Laser power and speed to keyholing, lack of fusion or balling, with process maps.	[6]
	`am-alloy-screening`	Known or new alloy to properties, a lack-of-fusion process map and a printability ranking.	[7]
	`sls-process-optimization`	Polymer SLS test results to the next batch's settings, batch by batch.	[8]
	`meltpool-geometry`	Melt-pool depth, width, length and mode from laser power, speed and alloy.	[34]
	`llm-3d-print-monitor`	Camera inspection of every layer and bounded in-print corrections on Klipper printers.	[9]
	`thermal-porosity`	Thermal images recorded during printing to pore count and location for each layer.	[35]
	`lpbf-superres`	Low-cost webcam images to high-resolution layer images and edge roughness.	[32]
	`am-literature-rag`	Page-cited answers from your own additive-manufacturing paper library.	[23]
Teaching			
	`worked-problem-generator`	Textbook-style problems with code-verified solutions.	–
	`mechanics-visuals`	Free-body diagrams, Mohr's circle, beam diagrams and S–N curves.	–

7 From paper to skill

We used the same procedure for every research skill:

1. **Extract the engineering workflow**, not the model. Most papers describe a model and a benchmark; a practitioner needs the steps around it—what to measure, how to prepare inputs, how to decide. We write those steps first.
2. **Re-derive the physics and build exact checks** before any learned component: a solid cell and a laminate for homogenization, Euler–Bernoulli bending for a stress generator, published OpenRocket predictions for flight.
3. **Reproduce the paper's headline experiment** with its protocol, and record the numbers in the tutorial.

4. **Exercise the workflow as a user would**: sanitize units on load, replay sequential decisions without future information, and confirm generated designs by re-simulation.

5. **Write the instructions last**, including the judgement calls discovered in steps 2–4, and add tests for every check.

When code or data were unavailable or impractical (several repositories were placeholders, one was 13 GB), the skill implements the method from the paper and ships a generator or rehearsal data so that the workflow can be exercised end to end.

8 Case studies

8.1 Lattice inverse design (GLU3D)

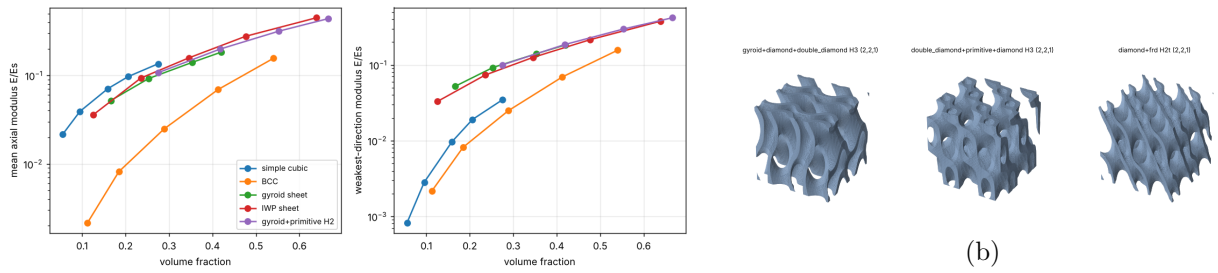
GLU3D [30] generates lattice unit cells from a target stiffness with a 3D conditional diffusion model trained on homogenized TPMS and hybrid-TPMS cells. The skill first builds the physics: implicit cells, printability filters (one periodic solid that reaches every face, no sealed cavities), and an energy-based periodic voxel homogenizer [48] that returns the full 6×6 stiffness. The homogenizer reproduces a solid cell exactly and matches the exact C_{33}, C_{44} and C_{11} of a two-phase laminate to 10^{-6}; a sheet gyroid at volume fraction 0.25 gives $E/E_s = 0.086$ with exact cubic symmetry. Figure 4(a) compares stiffness against simple-cubic and BCC strut lattices: at equal density the TPMS and hybrid sheets are about 5× stiffer than BCC and 3× stiffer than simple cubic in its weakest direction, though simple cubic is 1.3× stiffer along its own axes—which is why the skill reports the directional minimum.

For one-off designs the skill searches a homogenized dataset and refines thickness and level-set bias. For an anisotropic target $E_1 = E_2 = 0.06$, $E_3 = 0.15$ (relative to the solid) it returned a gyroid–diamond–double-diamond hybrid with $E = (0.057, 0.058, 0.151)$ at volume fraction 0.26 after re-checking at 48^3 (Fig. 4b), in two minutes on a laptop CPU.

The generative route reproduces the paper's training recipe. A literal reproduction on a small dataset (300 cells, 16^3, CPU) produced unstructured fields; four changes produced structured samples: training on the periodic grid with circular padding (which makes every generated cell exactly periodic), a sinusoidal-plus-MLP time embedding in place of a per-step lookup table, scaling the conditioning vector inside the encoder, and a longer learning-rate plateau patience. Stiffness error after re-homogenization remained 200–280% at this data scale, against 3723 cells at 64^3 and four GPUs in the paper; the skill therefore recommends search for production designs and accepts generated cells only after re-homogenization.

8.2 Printed rockets (RocketSmith)

RocketSmith [24] designed four FDM-printed Class 2 rockets that all flew stably; altimeter apogees were 80–84% of predictions. The skill reconstructs two of them from the paper and predicts apogee and speed within a few percent of OpenRocket: 326 m and 75 m/s against 338.8 m and 76.4 m/s for HP1, and 568 m and 113.7 m/s against 570.1 m and 115.2 m/s for HP4 (Fig. 5). Fitting the drag coefficient to the measured apogees (276 m and 479 m) gives $C_D \approx 0.84$ and 0.76, much higher than the 0.4–0.5 typical of smooth commercial airframes—a direct, actionable explanation for the shortfall: printed surfaces and fin geometry. The skill's fin-sizing loop also flags that HP1's fins (95 mm span) give marginal stability and computes the 162 mm span needed for 1.26 calibers.



(a) (b)

Figure 4: Lattice inverse design. (a) Mean axial and weakest-direction modulus against volume fraction for simple cubic, BCC, sheet gyroid, sheet I-WP and a gyroid–primitive hybrid, from the skill's homogenizer. (b) The three best cells returned by search for $E_1 = E_2 = 0.06$, $E_3 = 0.15$; anisotropy comes from two periods along x and y.

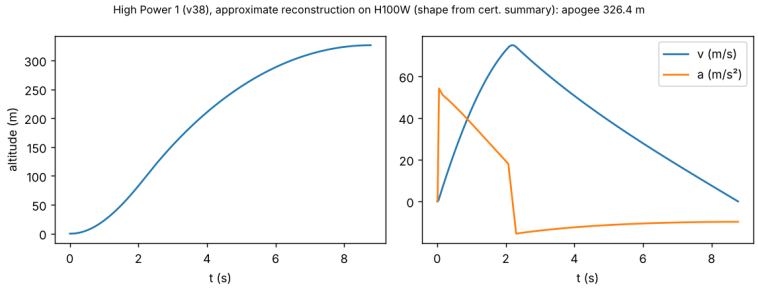


Figure 5: `rocketsmith`: simulated flight of the reconstructed HP1 rocket from the skill's Barrowman stability model and one-dimensional flight simulator; apogee is within 4% of OpenRocket.

8.3 Porosity from in-situ thermal images (ThermoPore)

Qualifying a laser powder-bed fusion part usually waits for X-ray computed tomography (CT). ThermoPore [35] asks how much of that answer is already contained in the pyrometry recorded during the build [52]. For each 30 μm layer it pairs 200 thermal frames with the CT pores in that layer and trains a CNN that counts pores ($R^2 = 0.57$, RMSE 7.84) and a video vision transformer [53, 54] that localizes them on a 16×16 grid (mean IoU 0.32). The skill packages the workflow around the models, which is where most practical effort goes: empty-frame filtering, resampling each layer to 200 frames, alignment of CT and thermal coordinates with an explicit check image, equivalent-diameter thresholds for which pores count, and splits within each process step as in the paper (Fig. 6). It adds an interpretable baseline—gradient-boosted trees on per-cell peak temperature, dwell time above the melt threshold and thermal roughness—that every deep model must beat. Because the paper's pyrometry and CT data are not public, the skill ships a synthetic rehearsal build with the same structure; on it the baseline reaches $R^2 = 0.44$ for counting and IoU 0.27 for localization, and every output from synthetic data is labelled as such so that it is never mistaken for a physical result. With a user's own build, the agent can simply be asked which layers and regions should be sent to CT first.

8.4 Sequential process optimization (SLS)

For polymer selective laser sintering [8], the skill keeps a build log of 21 published batches across three materials, reduces ASTM D638/D790 tests, and suggests the next batch with a bounded

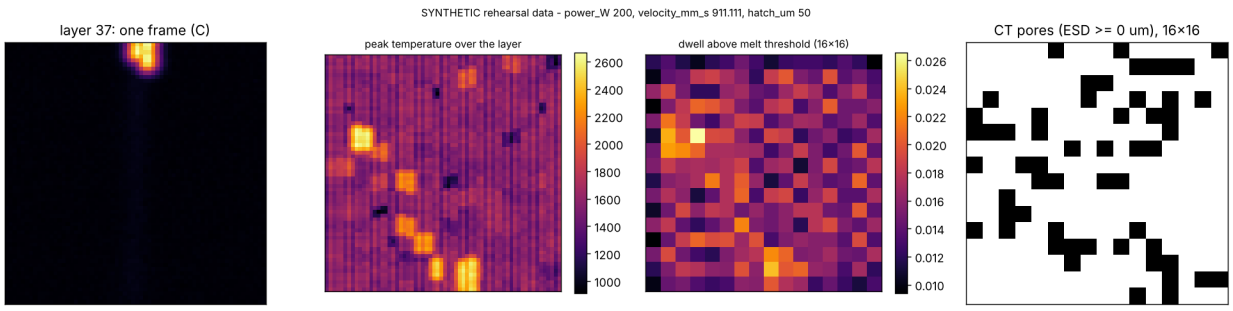


Figure 6: `thermal-porosity`: one layer of a rehearsal build. From left: a single pyrometry frame with the melt pool mid-raster; peak temperature over the layer, where hot and cold spots mark fusion anomalies; dwell time above the melt threshold pooled to the 16×16 label grid; and the CT pore map used as ground truth.

Gaussian-process expected-improvement step. To test it fairly we replayed the campaign, showing the suggester only the batches available at each point. After batches A–C it proposed raising hotspot overlap to 50%—what batch D did; after A–E it proposed fill energy 28 mJ/mm—batch I; after A–G, 30 mJ/mm. The suggestions are bounded to small steps and refuse surface temperatures within 3 °C of the melt.

8.5 Equation discovery (LLM-SR)

LLM-SR [40] lets a model propose equation skeletons as code while an optimizer fits constants. The skill runs proposals in an isolated interpreter after static safety checks, fits with BFGS from multiple starts, and scores in-domain and out-of-domain splits. On the stress–strain benchmark the best discovered skeleton reached normalized MSE 0.021/0.025/0.074 (train/in-domain/out-of-domain) against 0.161/0.150/0.680 for a cubic polynomial and 0.38 for linear regression; the advantage is largest where it matters for engineering, out of domain.

9 Limitations and future work

The library has been validated by tests and by reproducing published results, not yet by a controlled study of engineers using it; we plan task-based evaluations comparing agents with and without skills on design, analysis and manufacturing problems. Several skills need external software (OpenFOAM, FreeCAD, Thermo-Calc) or hardware (a Klipper printer, a powder-bed machine); they ship validators, mock devices or rehearsal data, but real use requires the tools. Learned components were rehearsed at small scale on CPU; their headline accuracy depends on retraining at the papers' scale. Finally, skills inherit the limits of their methods; the instructions state them, but they cannot replace an engineer's sign-off on safety-critical decisions.

10 Conclusion

Mechanical Engineering Skills packages 32 engineering capabilities, 19 of them from our research, in a form that a general agent can discover, load and use correctly. The central idea is simple: let the model plan and explain, let tested code compute, and return every result with its check. Because the interface is plain language, the whole library is available by text or voice, without learning each tool, training models or running them locally; and applying this discipline to our own papers reproduced

their key results, which suggests that a skill is also a useful standard for research reproducibility. The library is open-source under the MIT license; we invite contributions of skills from other groups.

Availability. Code, tests and tutorials:
https://github.com/BaratiLab/mechanical-engineering-skills;
website: https://baratilab.github.io/mechanical-engineering-skills/.
Install all skills with `npx skills add BaratiLab/mechanical-engineering-skills -all`.

References

[1] J. Barkley, R. Loghmani, and A. Barati Farimani. CADSmith: Multi-agent CAD generation with programmatic geometric validation. *arXiv preprint arXiv:2603.26512*, 2026.

[2] A. Badagabettu, S. S. Yarlagadda, and A. Barati Farimani. Query2CAD: Generating CAD models using natural language queries. *arXiv preprint arXiv:2406.00144*, 2024.

[3] Y. Jadhav and A. Barati Farimani. LinkD: AutoRegressive diffusion model for mechanical linkage synthesis. *arXiv preprint arXiv:2601.04054*, 2026.

[4] Y. Jadhav and A. Barati Farimani. Large language model agent as a mechanical designer. *Journal of Engineering Design*, 2026. arXiv:2404.17525.

[5] A. S. Oza, R. F. Johnson, and A. Barati Farimani. CFD Forge: A physics-constrained agentic framework for autonomous CFD. *arXiv preprint arXiv:2610.14104*, 2026.

[6] P. Pak and A. Barati Farimani. AdditiveLLM: Large language models predict defects in metals additive manufacturing. *Additive Manufacturing Letters*, 14:100292, 2025. doi: 10.1016/j.addlet.2025.100292.

[7] P. Pak, A. Chandrasekhar, and A. Barati Farimani. Agentic additive manufacturing alloy evaluation. *Additive Manufacturing Letters*, 17:100355, 2026. doi: 10.1016/j.addlet.2026.100355.

[8] P. Pak, V. Alvarado, and A. Barati Farimani. AI agentic selective laser sintering process optimization. *arXiv preprint arXiv:2608.25928*, 2026.

[9] Y. Jadhav, P. Pak, and A. Barati Farimani. LLM-3D Print: Large language models to monitor and control 3D printing. *Additive Manufacturing*, 114:105027, 2025. arXiv:2408.14307.

[10] Anthropic. Equipping agents for the real world with Agent Skills. Anthropic Engineering blog, October 2025. URL https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills.

[11] Agent Skills. Agent skills: an open format for packaging agent capabilities, 2025. URL https://agentskills.io. Published as an open standard in December 2025.

[12] Barati Farimani Lab. Mechanical engineering skills. GitHub repository, 2026. URL https://github.com/BaratiLab/mechanical-engineering-skills.

[13] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.

[14] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

[15] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[16] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[17] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18:186345, 2024.

[18] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

[19] Anthropic. Introducing the Model Context Protocol, November 2024. URL https://www.anthropic.com/news/model-context-protocol.

[20] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes. Autonomous chemical research with large language models. *Nature*, 624:570–578, 2023.

[21] A. M. Bran, S. Cox, O. Schilter, C. Baldassari, A. D. White, and P. Schwaller. Augmenting large language models with chemistry tools. *Nature Machine Intelligence*, 6:525–535, 2024.

[22] L. Makatura, M. Foshey, B. Wang, et al. How can large language models help humans in design and manufacturing? *arXiv preprint arXiv:2307.14377*, 2023.

[23] A. Chandrasekhar, J. Chan, F. Ogoke, O. Ajenifujah, and A. Barati Farimani. AMGPT: A large language model for contextual querying in additive manufacturing. *Additive Manufacturing Letters*, 11:100232, 2024. doi: 10.1016/j.addlet.2024.100232.

[24] P. Pak, J. Barkley, R. Loghmani, D. Baich, A. Pamal, and A. Barati Farimani. RocketSmith: Agentic additive manufacturing of high-powered rockets. *arXiv preprint arXiv:2606.00097*, 2026.

[25] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, volume 33, pages 6840–6851, 2020.

[26] A. Q. Nichol and P. Dhariwal. Improved denoising diffusion probabilistic models. In *International Conference on Machine Learning (ICML)*, pages 8162–8171, 2021.

[27] J. Song, C. Meng, and S. Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations (ICLR)*, 2021.

[28] J. Ho and T. Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022.

[29] Y. Jadhav, J. Berthel, C. Hu, R. Panat, J. Beuth, and A. Barati Farimani. StressD: 2D stress estimation using denoising diffusion model. *Computer Methods in Applied Mechanics and Engineering*, 416:116343, 2023. doi: 10.1016/j.cma.2023.116343.

[30] Y. Jadhav, J. Berthel, C. Hu, R. Panat, J. Beuth, and A. Barati Farimani. Generative lattice units with 3D diffusion for inverse design: GLU3D. *Advanced Functional Materials*, 34:2404165, 2024. doi: 10.1002/adfm.202404165.

[31] Z. Wang, Y. Jadhav, P. Pak, and A. Barati Farimani. Image2Gcode: Image-to-G-code generation for additive manufacturing using diffusion-transformer model. *Additive Manufacturing Letters*, 17:100374, 2026. doi: 10.1016/j.addlet.2026.100374.

[32] F. Ogoke, S. Kalambettu Suresh, J. Adamczyk, D. Bolintineanu, A. Garland, M. Heiden, and A. Barati Farimani. Deep learning based optical image super-resolution via generative diffusion models for layerwise in-situ LPBF monitoring. *Additive Manufacturing*, 107:104790, 2025. doi: 10.1016/j.addma.2025.104790.

[33] O. Ronneberger, P. Fischer, and T. Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241, 2015.

[34] P. Akbari, F. Ogoke, N.-Y. Kao, K. Meidani, C.-Y. Yeh, W. Lee, and A. Barati Farimani. MeltpoolNet: Melt pool characteristic prediction in metal additive manufacturing using machine learning. *Additive Manufacturing*, 55:102817, 2022. doi: 10.1016/j.addma.2022.102817.

[35] P. Pak, F. Ogoke, A. Polonsky, A. Garland, D. S. Bolintineanu, D. R. Moser, M. Arnhart, J. Madison, T. Ivanoff, J. Mitchell, B. Jared, B. Salzbrenner, M. J. Heiden, and A. Barati Farimani. ThermoPore: Predicting part porosity based on thermal images using deep learning. *Additive Manufacturing*, 95:104503, 2024. doi: 10.1016/j.addma.2024.104503.

[36] T. DebRoy, H. L. Wei, J. S. Zuback, T. Mukherjee, J. W. Elmer, J. O. Milewski, A. M. Beese, A. Wilson-Heid, A. De, and W. Zhang. Additive manufacturing of metallic components – process, structure and properties. *Progress in Materials Science*, 92:112–224, 2018.

[37] W. E. King, H. D. Barth, V. M. Castillo, G. F. Gallegos, J. W. Gibbs, D. E. Hahn, C. Kamath, and A. M. Rubenchik. Observation of keyhole-mode laser melting in laser powder-bed fusion additive manufacturing. *Journal of Materials Processing Technology*, 214:2915–2925, 2014.

[38] M. Schmidt and H. Lipson. Distilling free-form natural laws from experimental data. *Science*, 324:81–85, 2009.

[39] S.-M. Udrescu and M. Tegmark. AI Feynman: A physics-inspired method for symbolic regression. *Science Advances*, 6:eaay2631, 2020.

[40] P. Shojaee, K. Meidani, S. Gupta, A. Barati Farimani, and C. K. Reddy. LLM-SR: Scientific equation discovery via programming with large language models. In *International Conference on Learning Representations (ICLR)*, 2025. Oral; arXiv:2404.18400.

[41] D. Rosenthal. The theory of moving sources of heat and its application to metal treatments. *Transactions of the American Society of Mechanical Engineers*, 68:849–866, 1946.

[42] J. S. Barrowman. The practical calculation of the aerodynamic characteristics of slender finned vehicles. M.s. thesis, The Catholic University of America, 1967.

[43] H. G. Weller, G. Tabor, H. Jasak, and C. Fureby. A tensorial approach to computational continuum mechanics using object-oriented techniques. *Computers in Physics*, 12:620–631, 1998.

[44] S. Niskanen. Development of an open source model rocket simulation software. Master's thesis, Helsinki University of Technology, 2009.

[45] J.-O. Andersson, T. Helander, L. Höglund, P. Shi, and B. Sundman. Thermo-Calc & DICTRA, computational tools for materials science. *Calphad*, 26:273–312, 2002.

[46] L. J. Gibson and M. F. Ashby. *Cellular Solids: Structure and Properties*. Cambridge University Press, 2 edition, 1997.

[47] O. Al-Ketan and R. K. Abu Al-Rub. Multifunctional mechanical metamaterials based on triply periodic minimal surface lattices. *Advanced Engineering Materials*, 21:1900524, 2019.

[48] G. Dong, Y. Tang, and Y. F. Zhao. A 149 line homogenization code for three-dimensional cellular materials written in MATLAB. *Journal of Engineering Materials and Technology*, 141 (1):011005, 2019.

[49] D. R. Jones, M. Schonlau, and W. J. Welch. Efficient global optimization of expensive black-box functions. *Journal of Global Optimization*, 13:455–492, 1998.

[50] M. F. Ashby. *Materials Selection in Mechanical Design*. Butterworth-Heinemann, 4 edition, 2011.

[51] E. George, C. Keefe, P. Pak, and A. Barati Farimani. AgentsCAD: Automated design for manufacturing of FDM parts via multi-agent LLM reasoning and geometric feature recognition. *arXiv preprint arXiv:2607.02448*, 2026.

[52] S. K. Everton, M. Hirsch, P. Stravroulakis, R. K. Leach, and A. T. Clare. Review of in-situ process monitoring and in-situ metrology for metal additive manufacturing. *Materials & Design*, 95:431–445, 2016.

[53] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.

[54] A. Arnab, M. Dehghani, G. Heigold, C. Sun, M. Lučić, and C. Schmid. ViViT: A video vision transformer. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 6836–6846, 2021.